

Classification Metrics

Approach

In this assignment, `penguin_predictions.csv` will be used, where predictions are given for a female class in terms of probability, prediction label, and true label. To start with, the null error rate and the distribution of the classes will be plotted, which will also indicate the accuracy of the model. After that, confusion matrices will be made using three different probability values (0.2, 0.5, 0.8). These three different probability values will lead to three confusion matrices, and based on the number of true positives, true negatives, false positives, and false negatives obtained from these three matrices, accuracy, precision, recall, and F1 score will be calculated, and all of these will be shown in one table. The main challenge in this assignment is ensuring that the correct comparison between probability and threshold is made, as an incorrect comparison will affect all later calculations.

Code Deliverable

```
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':
```

```
filter, lag
```

```
The following objects are masked from 'package:base':
```

```
intersect, setdiff, setequal, union
```

```
library(ggplot2)
```

```
library(knitr)
```

```
df <- read.csv("penguin_predictions.csv")
```

```
n <- nrow(df)
```

Null Error Rate

The null error rate is the percentage of error you would get by always guessing the majority class.

```
class_counts <- table(df$sex)
majority_class <- names(which.max(class_counts))
majority_count <- max(class_counts)

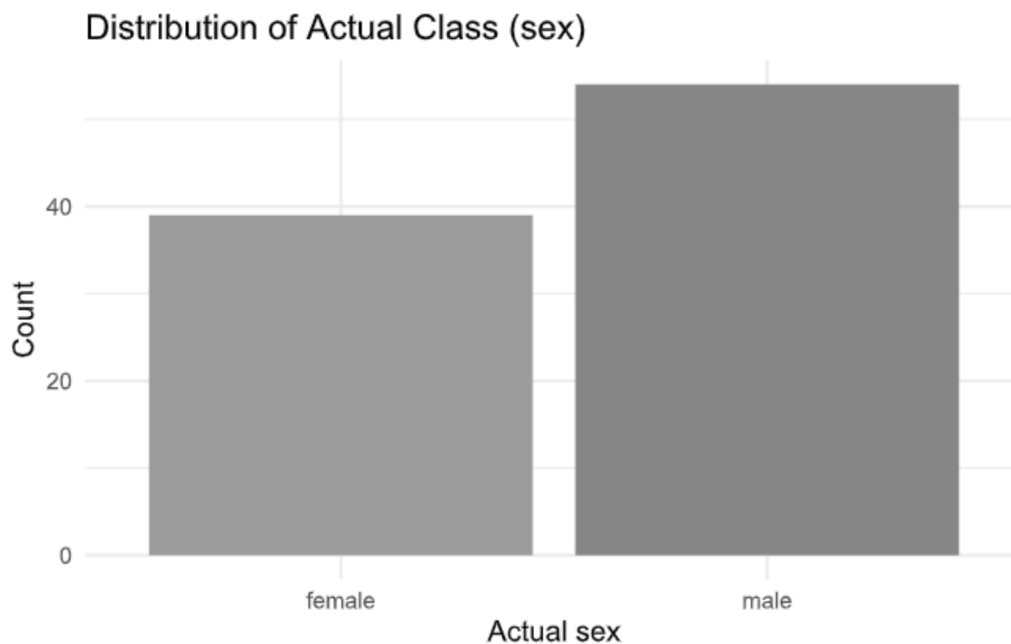
null_error_rate <- 1 - majority_count / n

null_error_rate
```

```
[1] 0.4193548
```

From running the operations above, the male is the majority class and the null error rate is 0.419.

```
ggplot(df, aes(x = sex, fill = sex)) +
  geom_bar() +
  labs(title = "Distribution of Actual Class (sex)",
       x = "Actual sex", y = "Count") +
  theme_minimal() +
  theme(legend.position = "none")
```



The null error rate matters because it's the baseline accuracy you'd get by always guessing the majority class, so it tells you how effective your model really is. Here, that baseline is 58.1% (a null error rate of 41.9%), which also signals some class imbalance in the data. Raw accuracy can be misleading on its own, since a model could score well just by leaning toward the majority class.

Confusion Matrices at Multiple Thresholds

A confusion matrix is a simple table that shows how a model's predictions line up against the actual outcomes for a binary classifier.

It has four cells:

- **True Positive (TP)** – predicted positive, actually positive
- **False Positive (FP)** – predicted positive, actually negative
- **True Negative (TN)** – predicted negative, actually negative
- **False Negative (FN)** – predicted negative, actually positive

In this assignment, “positive” = female. So TP is a penguin correctly predicted female, FP is a penguin predicted female that's actually male, and so on. The confusion matrix is the raw material everything else gets calculated from.

A penguin is predicted female whenever `.pred_female` is greater than the threshold.

```
actual_pos <- df$sex == "female"

# The three thresholds to evaluate
thresholds <- c(0.2, 0.5, 0.8)
```

We will create three matrices for each threshold by looping over them.

```
for (t in thresholds) {
  pred_pos <- df$.pred_female > t

  TP <- sum(pred_pos & actual_pos)    # predicted female, actually female
  FP <- sum(pred_pos & !actual_pos)   # predicted female, actually male
  TN <- sum(!pred_pos & !actual_pos)  # predicted male, actually male
  FN <- sum(!pred_pos & actual_pos)   # predicted male, actually female

  cat(sprintf("\nThreshold = %f\n", t))

  # Arrange TP/FN/FP/TN into a labeled 2x2 matrix for display
  c_matrix <- matrix(c(TP, FN, FP, TN), nrow = 2, byrow = TRUE,
                    dimnames = list(Actual = c("Female", "Male"),
                                     Predicted = c("Female", "Male")))

  print(c_matrix)
}
```

```
Threshold = 0.200000
```

	Predicted	
Actual	Female	Male
Female	37	2
Male	6	48

Threshold = 0.500000

	Predicted	
Actual	Female	Male
Female	36	3
Male	3	51

Threshold = 0.800000

	Predicted	
Actual	Female	Male
Female	36	3
Male	2	52

Performance Metrics

Performance metrics summarize how well a model does using the confusion matrix counts:

- **Accuracy** — overall % correct: $(TP+TN)/total$
- **Precision** — of predicted positives, how many were right: $TP/(TP+FP)$
- **Recall** — of actual positives, how many were caught: $TP/(TP+FN)$
- **F1** — balance of precision and recall: $2 \times (P \times R) / (P + R)$

```
actual_pos <- df$sex == "female"
thresholds <- c(0.2, 0.5, 0.8)

get_metrics <- function(threshold) {
  pred_pos <- df$.pred_female > threshold
  TP <- sum(pred_pos & actual_pos) # predicted female, actually female
  FP <- sum(pred_pos & !actual_pos) # predicted female, actually male
  TN <- sum(!pred_pos & !actual_pos) # predicted male, actually male
  FN <- sum(!pred_pos & actual_pos) # predicted male, actually female
  accuracy <- (TP + TN) / n # overall fraction correct
  precision <- TP / (TP + FP) # of predicted females, how many really are
  recall <- TP / (TP + FN) # of actual females, how many were caught
  f1 <- 2 * precision * recall / (precision + recall)

  # Package everything into a one-row data frame so results can be rbind'ed
  # together
  data.frame(threshold = threshold, TP = TP, FP = FP, TN = TN, FN = FN,
```

```

        accuracy = accuracy, precision = precision,
        recall = recall, f1 = f1)
}

# Evaluate at each threshold and stack the results into one table
results <- do.call(rbind, lapply(thresholds, get_metrics))

kable(results, digits = 4,
      col.names = c("Threshold", "TP", "FP", "TN", "FN",
                    "Accuracy", "Precision", "Recall", "F1 Score"))

```

Threshold	TP	FP	TN	FN	Accuracy	Precision	Recall	F1 Score
0.2	37	6	48	2	0.9140	0.8605	0.9487	0.9024
0.5	36	3	51	3	0.9355	0.9231	0.9231	0.9231
0.8	36	2	52	3	0.9462	0.9474	0.9231	0.9351

As the threshold rises from 0.2 to 0.8, precision climbs steadily with fewer false positives while recall stays roughly flat, which is normal.

Threshold Use Cases

Lower threshold (0.2) is preferred in case such as clinical application, such as tagging patients who should receive a cancer test. It's better to have a higher chance of making unnecessary tests due to false positives than risk not making the actual tests due to false negatives—recall takes precedence over precision.

Higher threshold (0.8) is preferred in a case such as a spam filter in a business email account. Only in this case would you need to make the classifier confident enough that the message is spam to tag it as such because a false positive (an important email that ends up in spam) is more undesirable than false negatives. A few spam emails are okay.

Conclusion

The model's accuracy significantly surpasses the “random guessing of the majority class” baseline (41.9%) at each possible threshold. Therefore, we can be confident that this classification model is not simply relying on the majority class advantage, but it is capturing some patterns in the input data. Moving from threshold 0.2 to 0.8 reduces the rate of false predictions (precision increases) while keeping the number of correct predictions nearly equal (recalls does not change dramatically). Threshold 0.5 is fine, but 0.8 performs slightly better in the current dataset. It's hard to say which threshold should be selected since “best” here depends solely on the specific use case and trade-offs between the cost of false positive/negative cases.

To further elaborate and validate the findings, it would be helpful to test additional thresholds beyond those examined here, such as every 0.05 or 0.1, and visualize how they affect the metrics

rather than observing only three time points. Creating the ROC curve and calculating the AUC are other ways to assess the classifier's performance without being biased by the choice of a single threshold. Testing this model on additional data or even using another sample altogether would prove the reliability of the findings and not confine them to this particular 93-observation sample. Lastly, testing another model, say, logistic regression instead of a decision tree, would help to determine whether the precision-recall relationship is specific for this particular model.